arXiv:2506.08947v1 [quant-ph] 10 Jun 2025

SCALING PORTFOLIO DIVERSIFICATION WITH QUANTUM CIRCUIT CUTTING TECHNIQUES

Vicente P. Soloviev
Fujitsu Research of Europe
vicente.perezsoloviev@fujitsu.com

Antonio Márquez Romero
Fujitsu Research of Europe
antonio.marquezromero@fujitsu.com

Josh Kirsopp
Fujitsu Research of Europe
josh.kirsopp@fujitsu.com

Michal Krompiec
Fujitsu Research of Europe
michal.krompiec@fujitsu.com

June 13, 2025

ABSTRACT

Quantum Approximate Optimization Algorithms (QAOA) have demonstrated a strong potential in addressing graph-based optimization problems. However, the execution of large-scale quantum circuits remains constrained by the limitations of current quantum hardware. In this work, we introduce `QuantCut`, an automatic framework for circuit cutting that enables efficient execution of large quantum circuits by decomposing entangling two-qubit gates into manageable sub-circuits. Specifically, we focus on gate-cutting techniques. We apply `QuantCut` to a 71-qubit QAOA circuit *ansatz* for portfolio diversification in the S&P 500 stock market, aiming to maximize asset diversification. Our approach iteratively optimizes the expectation value while leveraging circuit-cutting strategies to reduce the qubit register size. To validate our framework, we first conduct experiments on a toy model using quantum noise simulations for the Max-Cut problem, analyzing performance improvements with an increasing number of layers. Subsequently, we extend our methodology to a real-world financial optimization scenario, showing competitive results. The results suggest that `QuantCut` effectively facilitates large-scale quantum computations with circuit-cutting technologies.

Keywords QAOA · circuit cutting · knitting · portfolio · diversification · optimization

1 Introduction

Quantum Approximate Optimization Algorithms (QAOA) [Farhi et al., 2014] are widely used for graph-based optimization problems in the literature [Choi and Kim, 2019, Blekos et al., 2024]. QAOA involves optimizing a parameterized *ansatz* to minimize a desired objective expectation value. In the interest of reducing the runtime demands, we use circuit-cutting techniques to decrease the width of the measurement circuits.

Circuit cutting [Peng et al., 2020] consists in reproducing a quantum circuit by separating it into sub-circuits and reconstructing the final result with post-processing steps. Thus, each of the sub-circuits involves fewer qubits, allowing for larger computations on current quantum devices. Depending on how the original quantum circuit is partitioned, wire cutting and gate cutting have been proposed in the literature as distinct approaches.

Gate cutting consists of four main steps: (i) identifying the optimal gate cuts to perform in the circuit; (ii) generating a subset of circuit experiments whose qubit counts are reduced compared to the full circuit; (iii) executing the experiments; and (iv) reconstructing the final result.

In this paper, we propose `QuantCut`: an automatic framework for circuit cutting which automatically applies the aforementioned steps to a given quantum circuit. We demonstrate it with a proof-of-concept application to maximization

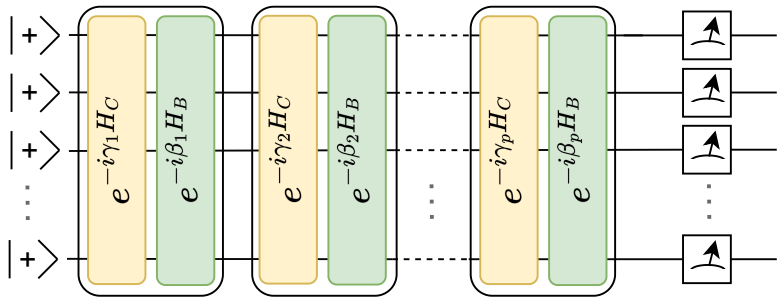


Figure 1: *ansatz* example with p layers and $2p$ parameters. The quantum circuit is initialized from a superposition state over all possible computational basis states, then p layers are applied, and all qubits are measured in the Z basis.

of the portfolio diversification among a set of assets selected from the S&P 500 stocks [1] with the QAOA approach by optimizing the expectation value of a 71-qubits *ansatz*.

This paper is structured as follows: Section 2 introduces some of the concepts and literature about QAOA, circuit cutting and portfolio diversification; Section 4 introduces `QuantCut`, how the QAOA problem is decomposed and the data-engineering process for the data taken from the S&P 500 stock market; Section 5 presents the main results of the experimentation with real data and quantum noise; and Section 6 rounds the paper off with some further conclusions and future research lines.

2 Background

2.1 Quantum approximate optimization algorithm

Variational quantum algorithms are hybrid classical-quantum approaches widely known in the literature [Cerezo et al., 2021]. They involve three main ingredients: (i) an objective function to be minimized, (ii) a parametric quantum circuit (henceforth referred to as an *ansatz*) and (iii) a classical optimizer that iteratively manipulates the *ansatz*. Some examples are the Variational Quantum Eigensolver [Peruzzo et al., 2014, Tilly et al., 2022], and the Quantum Approximate Optimization Algorithm (QAOA).

QAOA [Farhi et al., 2014] was initially proposed for combinatorial optimization problems. It has since been widely applied to challenges that can be represented as graphs, using quantum computing to explore large solution spaces.

The QAOA *ansatz* is composed of n qubits and $p \in \mathbb{N}$ layers, internally composed by (i) a cost operator $U(H_C, \gamma)$ (Eq. 1) parameterized by $\gamma \in [0, 2\pi]$ encoding the optimization task to be solved (H_C), and (ii) the mixed operator $U(H_B, \beta)$ (Eq. 2) parameterized by $\beta \in [0, 2\pi]$, which represents a rotation in the X-axis of each qubit (σ_j^x).

$$U(H_C, \gamma) = e^{-i\gamma H_C} = \prod_{\alpha=1}^{m} e^{-i\gamma C_\alpha}, \tag{1}$$

where C_α is the cost function to be minimized, m the number of clauses that define the classical cost function to be optimized, and H_C represents the cost Hamiltonian.

$$U(H_B, \beta) = e^{-i\beta H_B} = \prod_{j=1}^{n} e^{-i\beta\sigma_j^x}, \tag{2}$$

where H_B represents the mixed Hamiltonian.

Figure 1 shows an *ansatz* example in which the qubits are initialized in a superposition of computational basis states, and iteratively applies p layers ($2p$ parameters).

2.2 Circuit cutting

Circuit cutting is a technique designed to divide large quantum circuits into smaller, more manageable subcircuits, at the cost of performing more quantum experiments and additional classical post-processing overhead [Peng et al., 2020].

[1] https://www.kaggle.com/datasets/camnugent/sandp500

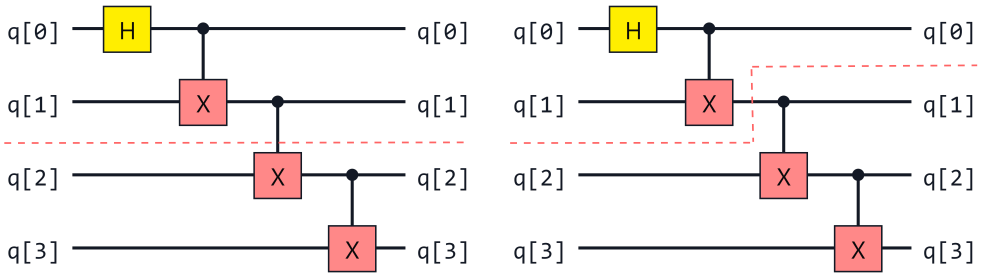


Figure 2: On the left, a gate-cutting (horizontal) cut applied to a GHZ circuit with four qubits. We observe that the second entangling CNOT is affected by the cut and its action will be reproduced in subsequent experiments in two subcircuits. On the right, a wire-cutting (vertical) cut is applied after the first entangling CNOT, the output quantum state will be saved and used as the initial state in the next circuit segment.

Such partitioning allows the quantum circuit to be run in different quantum devices in parallel, enabling distributed quantum computing [Caleffi et al., 2024, Barral et al., 2025, Tejedor et al., 2025]. Two flavors of cutting may be performed on a quantum circuit [Brandhofer et al., 2024]: wire- (also known as time-like, or vertical) and gate-cutting (also known as space-like, or horizontal). Their action on a quantum circuit can be seen in the diagram of Figure 2.

Wire cutting involves partitioning a quantum circuit by cutting through the wires, or qubit lines, that evolve the quantum information over time [Brenner et al., 2023]. This method partitions the circuit by breaking the temporal continuity of the state, effectively creating segments that can be processed separately, for different time steps.

Gate cutting, on the other hand, involves cutting through the entangling gates. This method partitions the circuit by breaking the qubit connections based on entanglement, resulting in smaller circuits where several experiments need to be run in order to reproduce locally the behavior of the entangling gates.

Both flavors incur an overhead scaling exponentially in terms of the number of cuts. In wire cutting, the overhead comes from the state preparation needed for the next circuit segment to be run, which is the measurement sampling result of the former segment. In gate cutting, the overhead comes from reproducing locally the non-local behavior of the entangling gates. In this work, we focus on the latter approach, and specifically for entangling gates involving two qubits only.

Any two-qubit entangling gate U can be represented as

$$U = [A_1 \otimes A_2] e^{i(a_X X \otimes X + a_Y Y \otimes Y + a_Z Z \otimes Z)} [B_1 \otimes B_2], \tag{3}$$

where X, Y, Z are the usual Pauli matrices and A_1, A_2, B_1, B_2 are one-qubit operators applied to numbered qubits 1 and 2, respectively. The interaction coefficients $[a_X, a_Y, a_Z]$ completely characterize the entangling gate. This representation is known as the KAK decomposition [Tucci, 2005]. We observe that, effectively, under this representation, cutting an entangling gate is equivalent to decomposing the *interaction* term given in the two-qubit exponential. In `QuantCut`, we follow the procedure proposed by Mitarai and Fujii to automatically cut an interaction operator, involving generating six different circuits and post-processing the measurement results in Eq. (B1) of [Mitarai and Fujii, 2021]. The channel $\mathcal{U}$ of the entangling unitary is reproduced by six different channels $\mathcal{F}$ involving only single-qubit gates

$$\mathcal{U} = \sum_{i=1}^{6} a_i \mathcal{F}_i, \tag{4}$$

where a_i are coefficients related to the parameters in the interaction terms in Eq. 3. This technique is called *quasiprobability decomposition* [Temme et al., 2017, Pashayan et al., 2015, Endo et al., 2018], in the sense that we sample the probability distribution of a non-local entangling gate by means of a linear combination of the sampling results of local operations.

We immediately observe that, if two or more entangling gates are to be cut, the post-measurement results need to be combined in order to reproduce the whole action of the entangling operators. Consequently, there is an exponential complexity as a function of the number of cuts involved in the circuit.

The most common entangling gate found in quantum circuits is the CNOT gate, or controlled-X, whose error rate is also used as a common metric for the quality of the quantum computer. It can be written as

$$\begin{aligned} CX &= e^{i\frac{\pi}{4}(I-Z)\otimes(I-X)} \\ &= e^{i\frac{\pi}{4}(II-IX-ZI+ZX)} \\ &= e^{i\frac{\pi}{4}}e^{-i\frac{\pi}{4}(IX+ZI)}e^{i\frac{\pi}{4}ZX} \\ &= e^{-i\frac{\pi}{4}}[S\otimes(HSH)]e^{i\frac{\pi}{4}YI}e^{i\frac{\pi}{4}XX}e^{-i\frac{\pi}{4}YI} \end{aligned} \tag{5}$$

where we have omitted the Kronecker product $\otimes$ for notation simplicity, and where H and S are the standard Hadamard and phase gates, respectively. This decomposition involves one-qubit gates only, except for the XX interaction term. Thus, we can follow the aforementioned recipe [Mitarai and Fujii, 2021] to cut the CNOT by cutting the interaction term $e^{i\frac{\pi}{4}XX}$. Further details may be found in Appendix A.

2.3 Portfolio diversification

Portfolio diversification is a fundamental concept in financial management, aimed at reducing the risk inherently associated with investment, by allocating funds across a variety of assets. By combining assets with different risk profiles, expected returns, and correlations, investors can create a portfolio that balances potential gains with risk exposure.

Accordingly, we define the profit return of a given portfolio as,

$$R_p = \sum_{i=1}^{n} w_i R_i, \tag{6}$$

where w_i and R_i are the weight and expected return of asset i, respectively.

The portfolio variance, which represents its associated risk, is weighted through the pairwise covariance as,

$$\sigma_p^2 = \sum_{i=1}^{n}\sum_{j=1}^{n} w_i w_j \mathrm{Cov}(R_i, R_j), \tag{7}$$

where $\mathrm{Cov}(R_i, R_j)$ is the covariance between the respective return of assets i and j.

Portfolio diversification aims to mitigate the risk by minimizing the portfolio variance between the assets according to Equation 7.

3 `QuantCut`: An automatic tool for circuit cutting

In this section, we present our framework for automatic circuit cutting. More specifically, `QuantCut` implements an automatic gate-cutting strategy. The tool has been implemented in order to be easily integrated in the pipeline of other applications. Thus, it can automatically break the original quantum circuit into optimal sub-circuits conforming to the qubit number limitations of a given quantum device or simulator, and posteriorly reconstruct the measurement result of the full circuit. Moreover, it allows the user to fine tune each of the steps of the process. The main modules in the framework are: (i) the cut finder, which automatically returns the optimal gate cuts to be performed given an initial quantum circuit and respective maximum number of qubits allowed; (ii) the post processing, which generates the respective experiments according to the cuts (Section 2.2) and posteriorly reconstructs the final result; and (iii) the plotting module for visualization of the results.

Figure 3 shows a flowchart describing the behavior of `QuantCut`. The inputs given by the user are (i) a quantum circuit to be cut, (ii) maximum number of qubits in subcircuits, (iii) a backend where the quantum circuits are executed (real quantum device or quantum simulator), and (iv) an (optional) operator to be measured on the quantum circuit. The following sections describe each of the cited modules in depth.

3.1 Automatic cut finder

Given an original quantum circuit, the automatic cut finder will return the optimal cuts to be performed in order to minimize the number of cuts while satisfying the quantum device restrictions.

Once the original quantum circuit is provided to the framework, manual and automatic cutting modes are available to the user. Each subcircuit generated by the framework acts upon a subset of qubits from the original quantum circuit. In

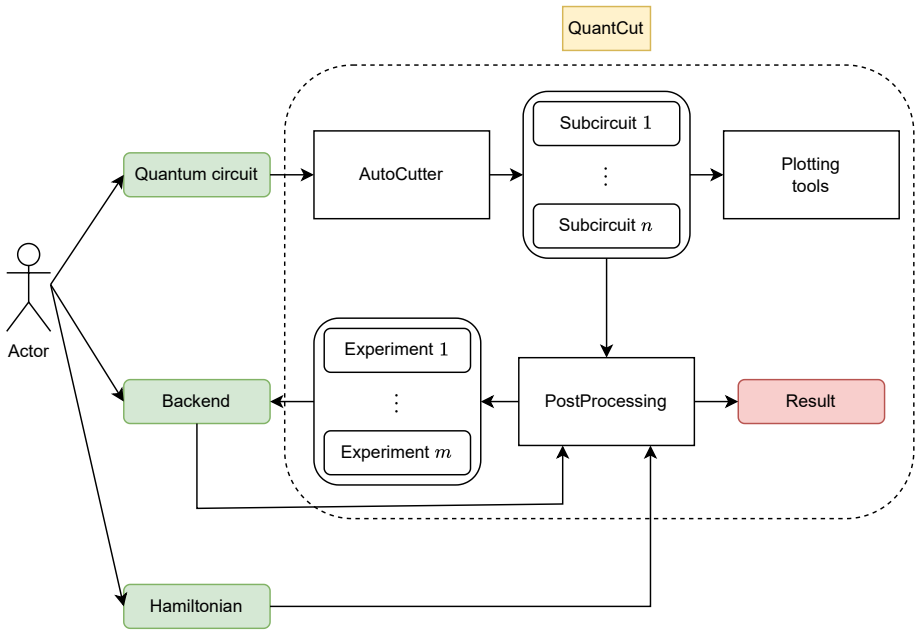


Figure 3: General flowchart of `QuantCut` where the three main modules are identified: (i) automatic cut finder (autocutter), (ii) post processing (postprocessing), and (iii) the plotting module (plotting tools).

order to perform circuit cutting, there must be at least two subcircuits with at least one qubit per subcircuit. The gates to be cut are those 2-qubit gates in which the control and target qubits are located in separate subcircuits. The more cuts to perform, the more complex the circuit-cutting procedure. Specifically, the number of experiments grows exponentially with the number of cuts. Thus, minimizing the number of cuts is critical.

In this module, we perform the optimal placement of cuts and minimization of their number. The qubits' connectivities are represented as binary variables x_{ij} where $x_{ij} = 0$ if there are no qubit gates between qubits i and j, and $x_{ij} = 1$ otherwise. Hence, the optimization problem is formulated as:

$$\text{minimize } f(x) \quad \text{subject to:}$$
$$f(x) = \begin{cases} \sum_{i,j}^{i>j} x_{ij} w_{ij} & \text{if } \texttt{isvalid}(x) \\ \delta & \text{otherwise} \end{cases}, \tag{8}$$

where $x_{ij} \in 0, 1$; w_{ij} denotes the number of 2-qubit gates between qubits i and j; $\texttt{isvalid}(x)$ is *True* if the solution generates the subcircuits satisfying the maximum number of qubits constraint provided by the user, and *False* otherwise; and δ is a penalization term. If a manual cut is requested instead of the automatic cut finder, then the positions of the gate cuts should be provided by the user.

`QuantCut` internally uses evolutionary computation to minimize the cost function. EDAspy [Soloviev et al., 2024] is used to implement a version of the estimation of distribution algorithm here. This module returns the subcircuits as a result of the optimization process. To the best of our knowledge, this is the first disclosed implementation of an automatic algorithm that finds the most suitable gate cuts for gate-cutting strategies based on qubit connectivity.

3.2 Post processing

This module generates a set of experiments from the given subcircuits and cuts (Section 3.1). Combining the results of these experiments, together with the given Hamiltonian or, generally, quantum operator to be measured, allows to reconstruct the final result. Note that the experiments are executed in a user-specified backend (quantum device or simulator) where the original circuit can not be executed due to the number of qubits available.

The Hamiltonian to be measured is split into subterms following the cuts given in the previous step. Then, the experiments are generated according to Section 2.2. The expectation value for each of the experiments is independently

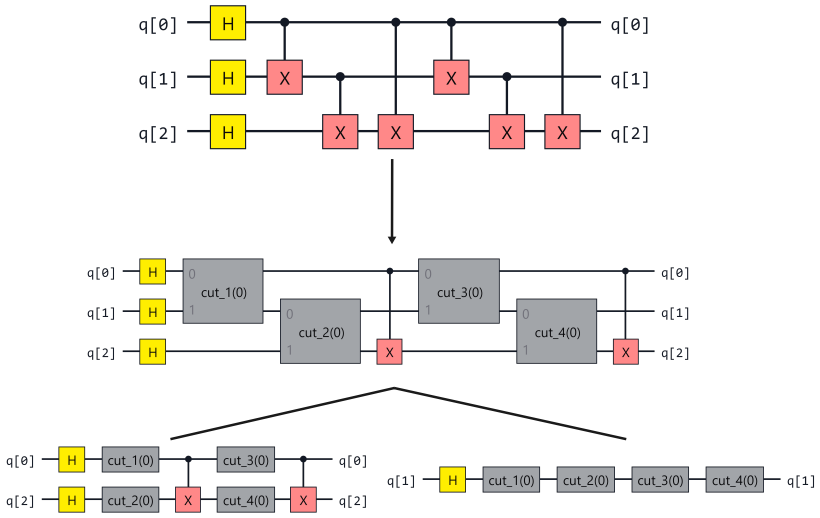


Figure 4: Example case in which an original quantum circuit is split into subcircuits of at most 2 qubits size.

computed. `QuantCut` parallelizes this computation over all the available CPUs (or QPUs, if available) to improve the runtime.

The framework provides not only the capability to return the reconstructed expectation value, but also the resulting state vector. Note that reconstructing the state vector is limited to a small number of qubits due to the exponential memory cost of storing the full state vector.

3.3 Plotting tool

As previously mentioned, our framework provides graphical visualization of circuit cuts. Using *pytket*'s rendering functionality [Sivarajah et al., 2020], we depict the gates to be cut as gray boxes in the original quantum circuit. A custom *pytket* gate is defined for this purpose, where each cut is labeled as `cut_i` within a gray box, while the control and target qubits retain their configuration from the original gate. When visualizing the subcircuits separately, the gate cuts are displayed as single-qubit operations within each subcircuit. The labels are duplicated and positioned identically in the quantum circuit representation of each subcircuit. Figure 4 shows an example in which an original quantum circuit is split into subcircuits of at most two qubits' size. Thus, the first subcircuit includes qubits "q[0]" and "q[2]", while the second subcircuit includes qubit "q[1]", respectively.

4 Method

In this Section we explain how `QuantCut` (Section 3) has been combined with the QAOA approach (Section 2.1) to target the portfolio diversification problem (Section 2.3).

The proposed approach follows the following pipeline:

1. Data engineering (see Appendix B for further details). The initial data is preprocessed.
2. Graph encoding (Section 4.1). The stock market is represented as a graph in which nodes and edges represent the assets and covariances between them, respectively.
3. Optimization task (Section 4.2). The QAOA circuit is constructed to solve the optimization problem.
4. Circuit cutting (Section 4.3). `QuantCut` is used to measure the original quantum circuit by performing circuit cutting.
5. Solution extraction (Section 4.4). After the optimal parameters are found, the optimal solution is extracted and analyzed.

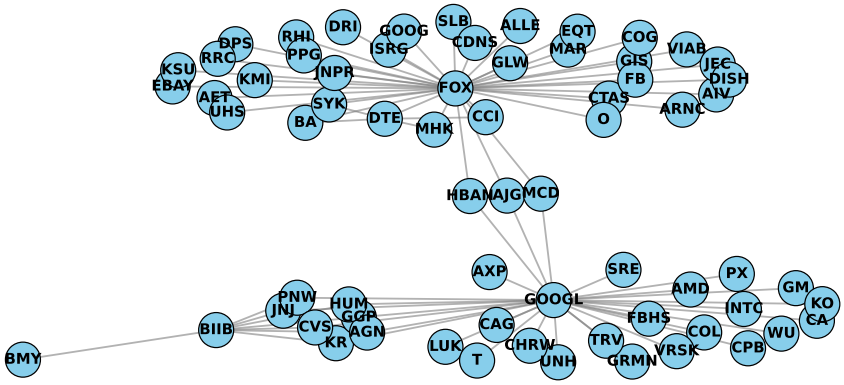


Figure 5: Stock market graph where each node represents an asset in the market, labeled by their corresponding ticker symbols, and edges between them represent the pairwise correlations.

4.1 Graph encoding

As described in Section 2.1, QAOA has been traditionally applied to solve graph optimization problems. Thus, in this subsection we describe how to graphically encode the portfolio optimization problem.

The stock market and its graphical representation has been deeply studied in the literature [Nagurney, 2003]. According to it, each node in the graph represents a stock asset in the market and the edges between the nodes represent the correlations in-between. In the interest of avoiding fully-connected graphs, the literature has proposed to impose a threshold α, so that correlations below α will not be represented in the graph. This thresholding approach results in a market graph that more accurately reflects the actual structure of the stock market, where not all assets are significantly correlated with each other.[Nagurney, 2003].

In this work we will use one node per asset represented. Thus, we are using a graph composed of 71 nodes. The threshold α has been empirically adjusted to $\alpha = 0.2$ over the covariance matrix (Section B). Figure 5 shows the stock market graph used in our approach.

4.2 Optimization problem

This section describes how the diversification problem (Section 2.3) represented as a graph as described in Section 4.1 is solved by the QAOA approach.

Since the objective of the diversification problem is to reduce redundancy between highly correlated assets, the optimization task consists of detecting those conflicting edges. To this end, we have mapped the task into the Max-Cut problem, where nodes connected by maximal weights have to be assigned to different portfolios.

The Max-Cut problem is NP-hard, which means that finding an exact solution for large graphs is computationally intensive and often impossible within a reasonable runtime. Approximation methods, heuristics, and algorithms such as semi-definite programming or quantum approaches (e.g. QAOA) are often employed to tackle it effectively. The optimization problem is defined as

$$\text{Max-Cut(G)} = \sum_{v_i, v_j \in E} w_{ij} x_i (1 - x_j), \tag{9}$$

where E is the set of edges in the graph G from nodes v_i to v_j and associated weight w_{ij}; and x_i and x_j are binary variables which assign the portfolio to nodes v_i and v_j, respectively. Since this is a binary optimization problem, the resulting solution of this process is the diversified strategy to locate the assets into two different portfolios. For more than two portfolios, the repeated bisection strategy [Dees et al., 2020] is used, in which the optimization task is repeatedly run over each of the sub-portfolios. In the extreme case where we infinitely many times run this strategy the result would be to assign each of the assets to a different portfolio.

As introduced in Section 2.1, the market graph is mapped into the cost operator (Eq. 1) and applied in each of the layers of the *ansatz*. In this approach, each node in the graph is assigned to a qubit in the system. There exist well-known strategies in the literature to map more than one node per qubit such as Pauli encoding [Sciorilli et al., 2025, Maciejewski et al., 2024]. However, this was out of the scope of this project and is proposed as a future research direction. Each of the edges in the graph involves either a combination of two (i) CX gates, or a (ii) CRZ gate between both qubits representing the respective nodes. In the interest of reducing the complexity of the circuit experiments in `QuantCut` we use the latter option in this approach.

4.3 Circuit cutting

This section describes how `QuantCut` is integrated in our QAOA pipeline, although it can be applied to other algorithms' pipelines.

For each parameter configuration in the *ansatz*, circuit cutting is applied over the entire quantum circuit in order to compute the expectation value. The automatic cut finder (Section 3.1) is only run in the first iteration, and the same gate cuts are applied in the successive iterations. The post processing step (Section 3.2) is run in each of the iterations.

4.4 Solutions extraction

Once we have an optimized quantum circuit, we would like to know the optimal solution, that is, the largest-amplitude qubit basis state. We could proceed by a straightforward measurement on all the wires, but, since we handle circuits with more than 70 qubits in this work, this task becomes unmanageable for a naive classical state vector simulation. An approximate solution may be found based on a dynamic-programming implementation of the marginal probabilities resulting from the measurement of subsets of qubits [Tang and Martonosi, 2022]. In this work, we opt for the simulation of the circuits using a tensor-network approach.

Tensor networks have proven to be practical in many scientific areas involving high-dimensional numerical simulations [Orús, 2014, Bañuls, 2023, García and Romero, 2024], including large quantum circuits [Vidal, 2003, Markov and Shi, 2008]. Broadly speaking, tensor networks are a collection of interconnected multidimensional matrices configured to obtain a quantity of interest through tensor contraction [Evenbly, 2022]. They are an efficient representation in terms of the minimal units describing the problem at hand, allowing to keep the number of floating-point operations involved to a minimum. They become especially useful when dealing with sparse or low-correlated systems. Approximations on tensor networks can be made, allowing an exponential complexity reduction at the expense of a well-controlled accuracy.

We illustrate how a quantum circuit can be represented, and manipulated, by a tensor network. Let us start with the standard initial state in a quantum circuit, $|0\rangle$. For a circuit with three qubits, a standard representation for this state as a tensor is

$$|000\rangle \equiv \mathbf{0}_{ABC}, \tag{10}$$

where A, B and C are physical indices needed to extract a single element out of this quantum state. As we have three qubits and three indices, the dimensions of these are 2 for all three. Graphically, this tensor can be depicted as in Fig. 6a. Now we apply the quantum circuit as described in Fig. 6b to this state. Again, this circuit can be depicted as a tensor network as shown in Fig. 6c. We observe six open indices: three for the input quantum state we have defined already in Eq. 10 and another three for the resulting state.

In terms of quantum operations, the resulting state is equivalent to

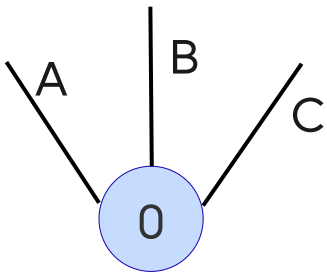
$$\begin{aligned} |\Psi\rangle &= CX_{(1,2)}CX_{(0,2)}H_0H_1H_2|000\rangle, \\ &\equiv \mathbf{\Psi}_{HIJ}, \end{aligned} \tag{11}$$

and, in terms of tensor networks, the resulting quantum state is

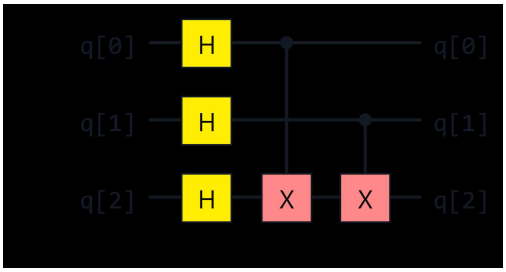
$$\mathbf{\Psi}_{HIJ} = CX_{DFGH}CX_{EFIJ}H_{AE}H_{BG}H_{CD}\mathbf{0}_{ABC}, \tag{12}$$

where we follow Einstein's convention of summation: repeated indices are to be summed. We see that two-qubit gates involve tensors with four indices while one-qubit gates only require two. This expression is obtained by *fusing* the tensor networks of the initial state in Fig. 6a and the quantum circuit in Fig. 6c.

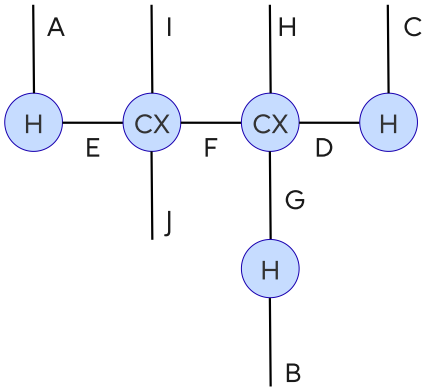
Thus, we observe how with the tensor-network formalism we are able to retrieve states from a quantum circuit. While for the aforementioned example we did not enforce it, there are specific tensor-network structures which allow a highly-optimized index contraction routine and an exponential reduction of computational resources needed to obtain such states under well-controlled approximations. One of the most popular representations is the *matrix product state* (MPS) [Perez-Garcia et al., 2006], valid for quantum circuits involving only neighboring entangling gates as shown



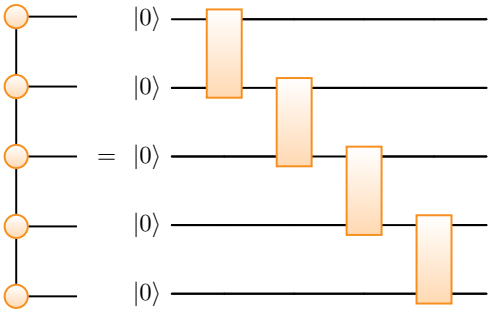
(a) Depiction of a tensor with three indices, representing the quantum state $|000\rangle$.



(b) Instance of quantum circuit, consisting of three Hadamard gates and two CNOTs.



(c) Tensor-network representation of the quantum circuit depicted in Fig. 6b. Indices shared upon two gates are to be summed.



(d) Matrix product state representation of a quantum circuit with neighboring entangling gates. Figure extracted from [García and Romero, 2024].

Figure 6: Tensor-network representation of quantum circuits and states.

in Fig. 6d. If a quantum circuit has non-neighboring entangling gates, it needs to be transformed by inserting SWAP gates to conform to nearest-neighbour coupling.. While this introduces an additional overhead in the circuit depth, it is compensated by making use of the polynomially-scaling MPS representation of the quantum circuit.

In `QuantCut`, we have used this tensor-network functionality as described above by means of the NVIDIA's *cuQuantum* [Bayraktar et al., 2023] integrated in *pytket* [Sivarajah et al., 2020] SDKs. In order to retrieve solutions, we sample the QAOA circuit and compute the associated amplitudes, extracting the solution with the largest amplitude. Calculations were made on a single NVIDIA A100 GPU.

5 Results and discussion

In this section, we firstly analyze the noise resilience of the approach with a toy example in which $n = 10$ qubits are used (Section 5.1), and secondly describe our results for portfolio diversification in which $n = 71$ qubits are used (Section 5.2).

5.1 Noise resilience

In this first experiment, we implement a toy example in which a random graph is generated and we solve the Max-Cut problem (Section 9) over it by using the QAOA. We use Erdős-Rényi approach [Erdos et al., 1960] to generate the initial graph with $n = 10$ nodes. In this experiment, we will simulate a noisy environment by considering the readout

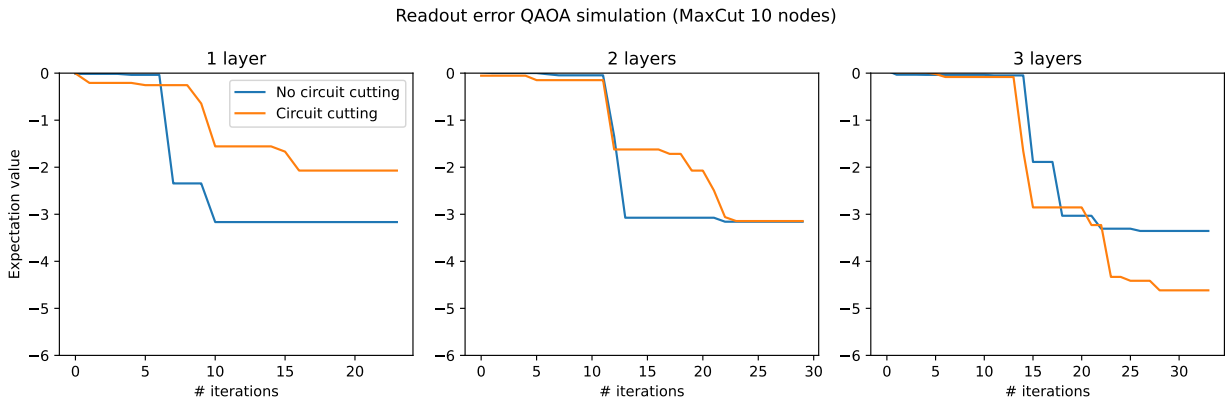


Figure 7: QAOA convergence plot with one, two and three layers, respectively, with (orange line) and without (blue line) circuit cutting.

noise model [Nielsen and Chuang, 2010] as,

$$\begin{pmatrix} P(0|0) & P(0|1) \\ P(1|0) & P(1|1) \end{pmatrix} = \begin{pmatrix} 0.99 & 0.01 \\ 0.01 & 0.99 \end{pmatrix}. \tag{13}$$

Figure 7 shows a comparison of the expectation value convergence during runtime of the QAOA approach for $p \in \{1, 2, 3\}$ layers in a noisy environment. Runs with and without circuit cutting are compared in a noisy simulated environment. In this case, one gate cut is performed per layer in the *ansatz*. A decreasing tendency of the converging expectation value is observed for an increasing number of layers in the QAOA *ansatz*, as described in the literature: the more layers in the *ansatz*, the better the expected performance. If the number of layers approaches infinity, the expectation value found is expected to be the optimal one.

Furthermore, a consistent improvement in circuit cutting accuracy is observed when compared to executions without cutting. However, with a single-layer configuration, the circuit cutting-based simulation demonstrates inferior convergence relative to its uncut counterpart. This trend reverses as the number of layers increases. These findings align with previous studies in the literature [Hakkaku et al., 2021] and suggest promising directions for investigating circuit cutting as a viable error-mitigation strategy.

5.2 71-qubit optimization

In this second experiment, we solve the problem of portfolio diversification for 71-qubits, where `QuantCut` is used in each iteration to compute the expectation value.

Figure 8 shows the convergence plot in which `QuantCut` performs three gate cuts for layer in the QAOA *ansatz*. Y- and X- axis represent the expectation value in each iteration and the runtime, respectively. A decreasing tendency is observed, showing that the classical optimizer is correctly minimizing the cost function. In this example, $p = 1$ layers were used. We now compare the results of our QAOA approach ($p = 1$) with random sampling and with optimization performed with a classical optimizer. In the latter case, we will use an evolutionary algorithm (EA) [Soloviev et al., 2024]. Table 1 shows a comparison of the cut edges found for the portfolio diversification problem (Eq. 9) as well as the edges in between the found sub-portfolios which is desired to be minimum:

$$\text{Acum}(G_i) = \sum_{v_i, v_j \in G_i} w_{ij}, \tag{14}$$

where G_i is a sub-portfolio generated as a result of the portfolio diversification problem over the original graph G, and w_{ij} each of the edges between the nodes in the subgraph representation.

By maximizing the former (Eq. 9) and minimizing the latter (Eq. 14), the portfolio diversification is maximized. It is observed that our approach outperforms the random sampling. However, the EA approach achieved better results than QAOA. Based on the results found in Section 5.1 and literature precedents [Farhi et al., 2014], we suppose that by increasing the number of layers (p) our approach will potentially converge to similar solutions as the ones found by the EA.

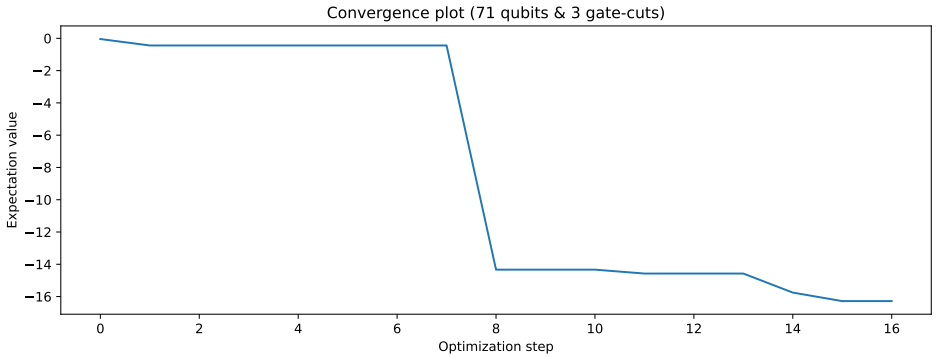


Figure 8: Convergence plot: expectation value minimization of the loss function in Eq. 9 along the iterative optimization process.

Table 1: Random sampling, QAOA ($p = 1$) and EA comparison of the Max-Cut(G) (Eq. 9) and Acum(G_i) (Eq. 14) metrics, where G_i are the resultant subgraphs after performing the cuts.

	Max-Cut(G)	Acum(G_0)	Acum(G_1)
Random sampling	26.1 ± 2.1	15.2 ± 5.0	7.9 ± 7.1
QAOA ($p = 1$)	43.41	2.439	4.113
EA	46.51	1.190	2.110

6 Conclusions

In this work, we present `QuantCut`: an automatic tool for circuit cutting. The tool is able to automatically find optimal locations of gate cuts by minimizing the number of cuts, and to perform post-processing calculations to reconstruct the measurement of the original circuit. `QuantCut` can be applied for the reconstruction of expectation values of operators and state vectors, although for the latter we must address the exponential memory-storage cost.

We iteratively use `QuantCut` to compute the expectation value of the QAOA *ansatz* during the optimization runtime. Firstly, we apply this pipeline to a simple example of a simulation with a quantum noise model, for the Max-Cut optimization problem, and analyze its behavior with respect to increasing the number of layers. Secondly, we solve a proof-of-concept problem in the finance domain: we maximize the diversification among selected assets from the S&P 500 stock market. In this work, we simulate up to 71-qubit quantum circuits. After optimization of the QAOA circuits, solutions in the form of a qubit-basis state can be obtained by sampling the circuits. Matrix product state tensor network representations of the circuits were used for optimal sampling performance. The numerical results demonstrate that our approach provides a competitive workflow for this problem, and may potentially increase resilience to quantum noise.

While this work has focused on QAOA and its applications to finance, we expect `QuantCut` to be a versatile tool with widespread applications where the simulation of large quantum circuits is involved. Specially interesting is the application of this automatic pipeline in the area of distributed quantum computing.

As future research work we propose to: (i) apply other problem mappings for assigning more than one asset per qubit; (ii) increase the number of layers in the QAOA *ansatz*; and (iii) analyze the performance of the approach and resilience of circuit cutting when different types of quantum noise are present.

A Cutting a CNOT

In this Appendix, we provide thorough details to perform the cut of the most common entangling gate found in quantum circuits: the controlled-NOT, or controlled-X, gate. It is well known that the CNOT, or CX can be written as

$$CX = e^{i\frac{\pi}{4}(I-Z)\otimes(I-X)}, \tag{15}$$

where we can develop the terms to find out that

$$\begin{aligned} CX &= e^{i\frac{\pi}{4}(I-Z)\otimes(I-X)} \\ &= e^{i\frac{\pi}{4}(II-IX-ZI+ZX)} \\ &= e^{i\frac{\pi}{4}}e^{-i\frac{\pi}{4}(IX+ZI)}e^{i\frac{\pi}{4}ZX} \\ &= e^{-i\frac{\pi}{4}}[S\otimes(HSH)]e^{i\frac{\pi}{4}YI}e^{i\frac{\pi}{4}XX}e^{-i\frac{\pi}{4}YI} \\ &= \frac{e^{-i\frac{\pi}{4}}}{2\sqrt{2}}[S\otimes(HSH)][(I+iY)\otimes I]e^{i\frac{\pi}{4}XX}[(I-iY)\otimes I] \end{aligned} \tag{16}$$

where we have at times omitted the Kronecker product $\otimes$ for the brevity of notation, and H and S are the standard Hadamard and phase gates, respectively. We have developed the expression to the minimal units separable in products of single-qubit gates applied to the action qubits, apart from the non-separable interaction operator $e^{i\frac{\pi}{4}XX}$.

Now, we can follow the aforementioned algorithm of reference [Mitarai and Fujii, 2021] to cut the CNOT by cutting the interaction operator, generating six different subexperiment circuits with only single-qubit gates. In addition, a practical guide into circuit cutting is also given on *Qiskit*'s explanatory material [Brańczyk et al., 2024], where they introduce an example for cutting the R_{ZZ} gate.

We summarize the aforementioned implementation using a general two-qubit unitary of the form

$$R = e^{i\theta A_1\otimes A_2}, \tag{17}$$

with A_i being any Pauli operator. We notice that the exponential sum in Eq. 3 can always be decomposed in the product of exponentials because the set of Pauli words XX, YY, ZZ commutes all with each other. The action of this gate on a two-qubit state generates sets of measurement counts which we denote by the general channel term $\mathcal{U}$. According to the algorithm, we have to generate six subexperiments to replicate this channel as Eq. 4 with only single-qubit gate channels $\mathcal{F}_i$. In this case, the coefficients a_i are given by

$$\begin{aligned} a_1 &= \cos(\theta)^2, \\ a_2 &= \sin(\theta)^2, \\ a_3 &= \sin(2\theta)/2, \\ a_4 &= -a_3, \\ a_5 &= a_3, \\ a_6 &= -a_3. \end{aligned} \tag{18}$$

We list now the six subexperiments needed:

1. We do nothing to the input circuit, we apply identity gates $I\otimes I$ and sum the contribution as
$$\mathcal{U} \longrightarrow \mathcal{U} + a_1\mathcal{F}_1. \tag{19}$$
We notice that $\mathcal{F}_1$ is the measurement result after the action of the two identity gates, that is, the quantum circuit without the entangling gate applied.
2. We apply the operators $A_1\otimes A_2$, count the results in $\mathcal{F}_2$ and add the contribution
$$\mathcal{U} \longrightarrow \mathcal{U} + a_2\mathcal{F}_2. \tag{20}$$
3. We apply the operators $M_{A_1}\otimes e^{i\frac{\pi}{4}A_2}$, where M_{A_1} is a projective measurement onto the A_1 basis. Thus, an additional classical register is needed for this subexperiment. Also, since we can only measure in the Z basis, if A_1 is X or Y, we need to rotate to this basis first, that is,
$$\begin{aligned} M_X &= HM_Z, \\ M_Y &= S^\dagger HM_Z. \end{aligned} \tag{21}$$
Associated with this measurement is the parameter β, with values 1 or -1 if the projective result of the first qubit is 0 or 1, respectively. We recollect the measurement results from this experiment in $\mathcal{F}_3$ and update as
$$\mathcal{U} \longrightarrow \mathcal{U} + \beta a_3\mathcal{F}_3. \tag{22}$$

4. Similar to experiment 3, but now with the gates $M_{A_1} \otimes e^{-i\pi/4A_2}$, and update

$$\mathcal{U} \longrightarrow \mathcal{U} + \beta a_4 \mathcal{F}_4. \tag{23}$$

5. We apply the operators $e^{i\frac{\pi}{4}A_1} \otimes M_{A_2}$, and update

$$\mathcal{U} \longrightarrow \mathcal{U} + \beta a_5 \mathcal{F}_5. \tag{24}$$

6. Similar to experiment 5, with gates $e^{-i\frac{\pi}{4}A_1} \otimes M_{A_2}$, and update

$$\mathcal{U} \longrightarrow \mathcal{U} + \beta a_6 \mathcal{F}_6. \tag{25}$$

After these six experiments, count recollection in $\mathcal{U}$ will be approximately the same as the original recollection from the two-qubit unitary action on the input qubit state. For the specific case of the CNOT, $\theta = \frac{\pi}{4}$, so the quasiprobabilities become $[a_1, a_2, a_3, a_4, a_5, a_6] = [0.5, 0.5, 0.5, -0.5, 0.5, -0.5]$. $A_1 = A_2 = X$ and thus we need to perform M_X measurements in steps 3-6.

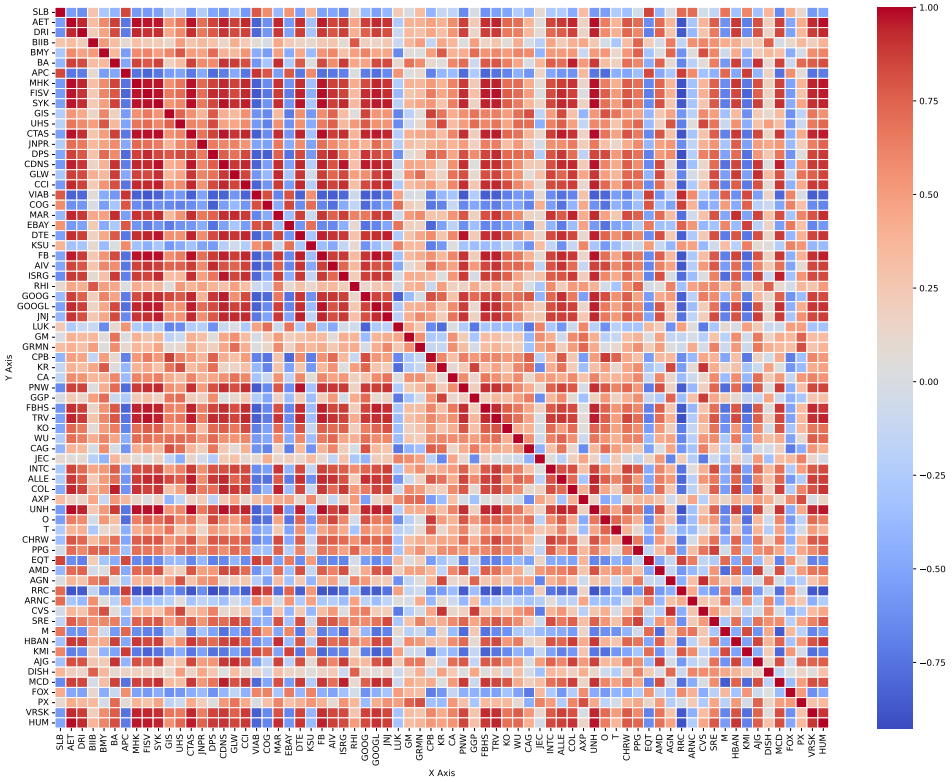


Figure 9: Correlation heatmap for all 71 assets considered in this work, where positive and negative correlations are represented as light and dark colors, respectively.

B Data engineering

For this experiment we will be using real data from the S&P 500 stock market [2], which collects the stock values from the last five years. In the interest of showcasing a real scenario, 71 assets out of 500 were carefully selected.

In this dataset we first analyze the correlation between the time series representing the time evolutions of the assets. Figure 9 shows a correlation heatmap among all the selected assets in which light and dark colors represent the positive and negative Pearson correlation coefficients, respectively, as

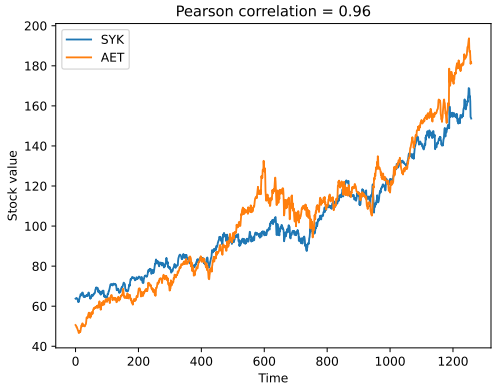
$$r = \frac{\sum(x_i y_i) - n\mu_x\mu_y}{\sqrt{\left(\sum x_i^2 - n\mu_x^2\right)\left(\sum y_i^2 - n\mu_y^2\right)}}, \tag{26}$$

where x_i and y_i represent the individual data points in two different datasets and μ_x and μ_y represent the mean values of both datasets.

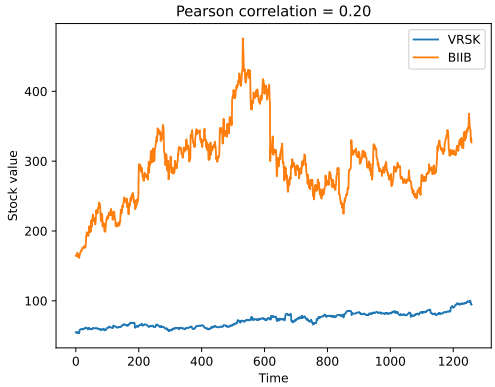
A positive correlation between two temporal time series of stock assets occurs when the movements of the two assets tend to align in the same direction over time. This means that as the value of one stock increases (decreases), the value of the other stock is likely to increase (decrease) in a similar manner. Figure 10 shows one example in which two assets follow a positive correlation along time (Subfigure 10a), and another example in which two assets are not correlated at all (Subfigure 10b).

In order to pre-process the time series, we normalize and standardize the data. Firstly, we normalize the data between 0 and 1 by scaling each each data point as $x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$ where $x_{\min}$ and $x_{\max}$ are the minimum and maximum values

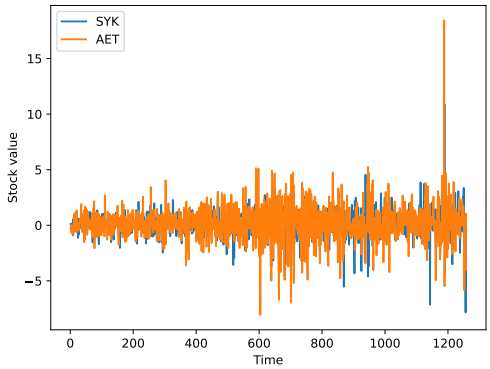
[2] https://www.kaggle.com/datasets/camnugent/sandp500



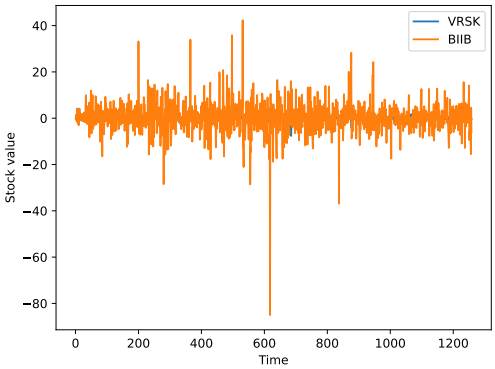
(a) Time series representing two stock values along time with positive correlation.



(b) Time series representing two stock values along time with low correlation.



(c) Rolling window time series representing two stock values along time with positive correlation.



(d) Rolling window time series representing two stock values along time with low correlation.

Figure 10: Time series examples plotting along time.

detected in the time series, respectively. Secondly, standardization transforms the data to have a mean of $\mu'' = 0$ and a standard deviation of $\sigma'' = 1$ by computing each data point as $x'' = \frac{x' - \mu'}{\sigma'}$, where μ' and σ' are the mean and standard deviation of the time series after the running the normalization, respectively.

After pre-processing the data series, we compute the covariance matrix over the standardized data, in which off-diagonal elements represent the pairwise variances among assets.

References

Mari Carmen Bañuls. Tensor network algorithms: A route map. *Annual Review of Condensed Matter Physics*, 14(1): 173–191, 2023. doi: https://doi.org/10.1146/annurev-conmatphys-040721-022705.

David Barral, F Javier Cardama, Guillermo Díaz-Camacho, Daniel Faílde, Iago F Llovo, Mariamo Mussa-Juane, Jorge Vázquez-Pérez, Juan Villasuso, César Piñeiro, Natalia Costas, et al. Review of distributed quantum computing: From single qpu to high performance quantum computing. *Computer Science Review*, 57:100747, 2025. doi: https://doi.org/10.1016/j.cosrev.2025.100747.

Harun Bayraktar, Ali Charara, David Clark, Saul Cohen, Timothy Costa, Yao-Lung L Fang, Yang Gao, Jack Guan, John Gunnels, Azzam Haidar, et al. cuquantum sdk: A high-performance library for accelerating quantum science. In *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*, volume 1, pages 1050–1061. IEEE, 2023. doi: https://doi.org/10.1109/QCE57702.2023.00119.

Kostas Blekos, Dean Brand, Andrea Ceschini, Chiao-Hui Chou, Rui-Hao Li, Komal Pandya, and Alessandro Summer. A review on quantum approximate optimization algorithm and its variants. *Physics Reports*, 1068:1–66, 2024.

Agata M. Brańczyk, Almudena Carrera Vazquez, Daniel J. Egger, Bryce Fuller, Julien Gacon, James R. Garrison, Jennifer R. Glick, Caleb Johnson, Saasha Joshi, Edwin Pednault, C. D. Pemmaraju, Pedro Rivero, Ibrahim Shehzad, and Stefan Woerner. Qiskit addon: circuit cutting. https://github.com/Qiskit/qiskit-addon-cutting, 2024.

Sebastian Brandhofer, Ilia Polian, and Kevin Krsulich. Optimal partitioning of quantum circuits using gate cuts and wire cuts. *IEEE Transactions on Quantum Engineering*, 5:1–10, 2024. doi: https://doi.org/10.1109/TQE.2023.3347106.

Lukas Brenner, Christophe Piveteau, and David Sutter. Optimal wire cutting with classical communication. *arXiv preprint arXiv:2302.03366*, 2023.

Marcello Caleffi, Michele Amoretti, Davide Ferrari, Jessica Illiano, Antonio Manzalini, and Angela Sara Cacciapuoti. Distributed quantum computing: A survey. *Computer Networks*, 254:110672, 2024. ISSN 1389-1286. doi: https://doi.org/10.1016/j.comnet.2024.110672.

Marco Cerezo, Andrew Arrasmith, Ryan Babbush, Simon C Benjamin, Suguru Endo, Keisuke Fujii, Jarrod R McClean, Kosuke Mitarai, Xiao Yuan, Lukasz Cincio, et al. Variational quantum algorithms. *Nature Reviews Physics*, 3(9): 625–644, 2021. doi: https://doi.org/10.1038/s42254-021-00348-9.

Jaeho Choi and Joongheon Kim. A tutorial on quantum approximate optimization algorithm (QAOA): Fundamentals and applications. In *2019 International Conference on Information and Communication Technology Convergence (ICTC)*, pages 138–142. IEEE, 2019. doi: https://doi.org/10.1109/ICTC46691.2019.8939749.

Bruno Scalzo Dees, Ljubiša Stanković, Anthony G Constantinides, and Danilo P Mandic. Portfolio cuts: A graph-theoretic framework to diversification. In *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 8454–8458. IEEE, 2020. doi: https://doi.org/10.1109/ICASSP40776.2020.9054371.

Suguru Endo, Simon C. Benjamin, and Ying Li. Practical quantum error mitigation for near-future applications. *Phys. Rev. X*, 8:031027, Jul 2018. doi: https://doi.org/10.1103/PhysRevX.8.031027.

Paul Erdos, Alfréd Rényi, et al. On the evolution of random graphs. *Publ. math. inst. hung. acad. sci*, 5(1):17–60, 1960. doi: https://doi.org/doi.org/10.1515/9781400841356.38.

Glen Evenbly. A practical guide to the numerical implementation of tensor networks i: Contractions, decompositions, and gauge freedom. *Frontiers in Applied Mathematics and Statistics*, 8:806549, 2022. doi: https://doi.org/10.3389/fams.2022.806549.

Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. A quantum approximate optimization algorithm. *arXiv preprint arXiv:1411.4028*, 2014.

Marcos Díez García and Antonio Márquez Romero. Survey on computational applications of tensor-network simulations. *IEEE Access*, 2024. doi: https://doi.org/10.1109/ACCESS.2024.3519676.

Shigeo Hakkaku, Kosuke Mitarai, and Keisuke Fujii. Sampling-based quasiprobability simulation for fault-tolerant quantum error correction on the surface codes under coherent noise. *Physical Review Research*, 3(4):043130, 2021.

Filip B Maciejewski, Bao G Bach, Maxime Dupont, P Aaron Lott, Bhuvanesh Sundar, David E Bernal Neira, Ilya Safro, and Davide Venturelli. A multilevel approach for solving large-scale qubo problems with noisy hybrid quantum approximate optimization. In *2024 IEEE High Performance Extreme Computing Conference (HPEC)*, pages 1–10. IEEE, 2024. doi: https://doi.org/10.1109/HPEC62836.2024.10938438.

Igor L Markov and Yaoyun Shi. Simulating quantum computation by contracting tensor networks. *SIAM Journal on Computing*, 38(3):963–981, 2008. doi: https://doi.org/10.1137/050644756.

Kosuke Mitarai and Keisuke Fujii. Constructing a virtual two-qubit gate by sampling single-qubit operations. *New Journal of Physics*, 23(2):023021, 2021. doi: https://doi.org/10.1088/1367-2630/abd7bc.

Anna Nagurney. Innovations in financial and economic networks. *OR/MS Today*, 30(6):60–61, 2003.

Michael A Nielsen and Isaac L Chuang. *Quantum computation and quantum information*. Cambridge university press, 2010.

Román Orús. A practical introduction to tensor networks: Matrix product states and projected entangled pair states. *Annals of physics*, 349:117–158, 2014. doi: https://doi.org/10.1016/j.aop.2014.06.013.

Hakop Pashayan, Joel J. Wallman, and Stephen D. Bartlett. Estimating outcome probabilities of quantum circuits using quasiprobabilities. *Phys. Rev. Lett.*, 115:070501, Aug 2015. doi: https://doi.org/10.1103/PhysRevLett.115.070501.

Tianyi Peng, Aram W. Harrow, Maris Ozols, and Xiaodi Wu. Simulating large quantum circuits on a small quantum computer. *Phys. Rev. Lett.*, 125:150504, Oct 2020. doi: https://doi.org/10.1103/PhysRevLett.125.150504.

David Perez-Garcia, Frank Verstraete, Michael M Wolf, and J Ignacio Cirac. Matrix product state representations. *arXiv preprint quant-ph/0608197*, 2006.

Alberto Peruzzo, Jarrod McClean, Peter Shadbolt, Man-Hong Yung, Xiao-Qi Zhou, Peter J Love, Alán Aspuru-Guzik, and Jeremy L O'brien. A variational eigenvalue solver on a photonic quantum processor. *Nature communications*, 5 (1):4213, 2014. doi: https://doi.org/10.1038/ncomms5213.

Marco Sciorilli, Lucas Borges, Taylor L Patti, Diego García-Martín, Giancarlo Camilo, Anima Anandkumar, and Leandro Aolita. Towards large-scale quantum optimization solvers with few qubits. *Nature Communications*, 16(1): 476, 2025. doi: https://doi.org/10.1038/s41467-024-55346-z.

Seyon Sivarajah, Silas Dilkes, Alexander Cowtan, Will Simmons, Alec Edgington, and Ross Duncan. t| ket>: a retargetable compiler for nisq devices. *Quantum Science and Technology*, 6(1):014003, 2020. doi: https://doi.org/10.1088/2058-9565/ab8e92.

Vicente P Soloviev, Pedro Larrañaga, and Concha Bielza. EDAspy: An extensible python package for estimation of distribution algorithms. *Neurocomputing*, 598:128043, 2024. doi: https://doi.org/10.1016/j.neucom.2024.128043.

Wei Tang and Margaret Martonosi. Cutting quantum circuits to run on quantum and classical platforms. *arXiv preprint arXiv:2205.05836*, 2022.

Mar Tejedor, Berta Casas, Javier Conejero, Alba Cervera-Lierta, and Rosa M Badia. Distributed quantum circuit cutting for hybrid quantum-classical high-performance computing. *arXiv preprint arXiv:2505.01184*, 2025.

Kristan Temme, Sergey Bravyi, and Jay M. Gambetta. Error mitigation for short-depth quantum circuits. *Phys. Rev. Lett.*, 119:180509, Nov 2017. doi: https://doi.org/10.1103/PhysRevLett.119.180509.

Jules Tilly, Hongxiang Chen, Shuxiang Cao, Dario Picozzi, Kanav Setia, Ying Li, Edward Grant, Leonard Wossnig, Ivan Rungger, George H Booth, et al. The variational quantum eigensolver: a review of methods and best practices. *Physics Reports*, 986:1–128, 2022. doi: https://doi.org/10.1016/j.physrep.2022.08.003.

Robert R Tucci. An introduction to Cartan's KAK decomposition for QC programmers. *arXiv preprint quant-ph/0507171*, 2005.

Guifré Vidal. Efficient classical simulation of slightly entangled quantum computations. *Physical review letters*, 91(14): 147902, 2003. doi: https://doi.org/10.1103/PhysRevLett.91.147902.